Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського» Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Організація баз даних

Лабораторна робота №5

«Нормалізація бази даних»

Виконала:

студентка групи ІО-46

Чубарева Д.Д.

Номер у списку групи: 16

Перевірив:

Русінов В. В.

Київ – 2026

Тема: «Нормалізація бази даних».

Цілі:

- Пошук надлишковості та аномалій: Виявлення потенційної надлишковості даних (наприклад, повторювані значення) або аномалій оновлення (проблеми вставки/оновлення/видалення) у поточній схемі.
- Перелік функціональних залежностей: Визначте та перелічіть функціональні залежності (ФЗ) для кожної проблемної таблиці.
- Перевірка нормальних форм: Оцініть поточну нормальну форму кожної таблиці (1NF, 2NF, 3NF) на основі її функциональних диференціалів (FD) та структури ключа.
- Застосування нормалізації: перетворення таблиць у вищі нормальні форми (до 3НФ) для усунення часткових та транзитивних залежностей.

Аналіз 1NF

У tournaments поле location містить такі значення, як "Лондон, Велика Британія" або "Буча, Україна", в одному полі зберігається відразу дві різні сутності: місто та країна, щоб це виправити, я розділю це поле на два окремих атрибути city та country.

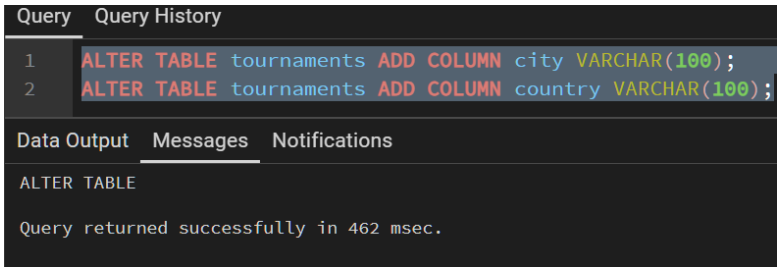
У matches score містить рахунок у форматі "3-1" або "2-3", це поле містить результати двох різних гравців, тут я розділю це поле на два окремих: player1_score та player2_score.

Виправлення

```
ALTER TABLE tournaments ADD COLUMN city VARCHAR(100);

ALTER TABLE tournaments ADD COLUMN country VARCHAR(100);
```

Спочатку створюємо два нових стовпці для tournaments




```
UPDATE tournaments SET city = 'Лондон', country = 'Велика Британія' WHERE
tournament_id = 1;

UPDATE tournaments SET city = 'Буча', country = 'Україна' WHERE
tournament_id = 2;

UPDATE tournaments SET city = 'Йокогама', country = 'Японія' WHERE
tournament_id = 3;
```

Тепер ми оновлюємо дані цих стовпців.

```
3  UPDATE tournaments SET city = 'Лондон', country = 'Велика Британія' WHERE tournament_id = 1
4  UPDATE tournaments SET city = 'Буча', country = 'Україна' WHERE tournament_id = 2;
5  UPDATE tournaments SET city = 'Йокогама', country = 'Японія' WHERE tournament_id = 3;
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 130 msec.
```

```
ALTER TABLE tournaments DROP COLUMN location;
```

Тепер видаляємо старий стовпець. Забула зробити скріншот успішного виконання, але на фото видно що старий стовпець вже не існує

```
6  ALTER TABLE tournaments DROP COLUMN location;
```

Data Output | Messages | Notifications

```
ERROR:  column "location" of relation "tournaments" does not exist

ОШИБКА:  column "location" of relation "tournaments" does not exist
SQL state: 42703
```

```
SELECT city, country FROM tournaments;
```

Тепер виконуємо перевірку

```
7  SELECT city, country FROM tournaments;
```

Data Output | Messages | Notifications

	city character varying (100)	country character varying (100)
1	Лондон	Велика Британія
2	Буча	Україна
3	Йокогама	Японія

```
ALTER TABLE matches ADD COLUMN player1_score VARCHAR(100);
ALTER TABLE matches ADD COLUMN player2_score VARCHAR(100);
```

Переходимо до виправлення у таблиці matches. Як і в tournaments, ми створюємо два нові стовпці

```
8  ALTER TABLE matches ADD COLUMN player1_score VARCHAR(100);
9  ALTER TABLE matches ADD COLUMN player2_score VARCHAR(100);
10
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 187 msec.
```

```
UPDATE matches SET player1_score = 3, player2_score = 1 WHERE match_id =
1;

UPDATE matches SET player1_score = 2, player2_score = 3 WHERE match_id =
2;

UPDATE matches SET player1_score = 3, player2_score = 0 WHERE match_id =
3;

UPDATE matches SET player1_score = 1, player2_score = 3 WHERE match_id =
4;

UPDATE matches SET player1_score = 0, player2_score = 3 WHERE match_id =
5;
```

Далі оновлюємо дані цих стовпців.

```
10 UPDATE matches SET player1_score = 3, player2_score = 1 WHERE match_id = 1;
11 UPDATE matches SET player1_score = 2, player2_score = 3 WHERE match_id = 2;
12 UPDATE matches SET player1_score = 3, player2_score = 0 WHERE match_id = 3;
13 UPDATE matches SET player1_score = 1, player2_score = 3 WHERE match_id = 4;
14 UPDATE matches SET player1_score = 0, player2_score = 3 WHERE match_id = 5;
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 175 msec.
```

```
ALTER TABLE matches DROP COLUMN score;
```

Тепер видаляємо старий стовпець.

```
15 ALTER TABLE matches DROP COLUMN score;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 327 msec.
```

```
SELECT player1_score, player2_score FROM matches;
```

Тепер виконуємо перевірку

```
16 SELECT player1_score, player2_score FROM matches;
```

Data Output | Messages | Notifications

	player1_score character varying (100)	player2_score character varying (100)
1	3	1
2	2	3
3	3	0
4	1	3
5	0	3

Аналіз 2NF

Оскільки у всіх таблицях бази використовуються прості одиничні первинні ключі, часткові залежності відсутні повністю. База даних автоматично знаходиться у 2NF. Отже виконувати нормалізацію немає потреби.

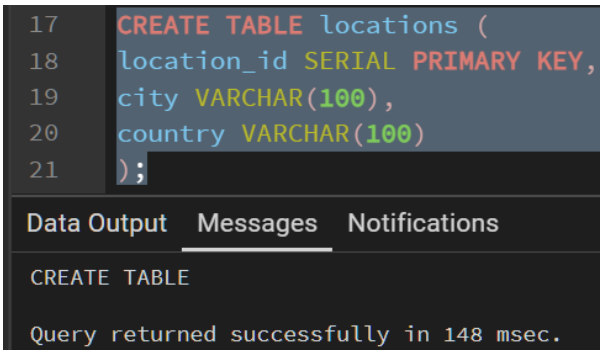
Аналіз 3NF

Після оновлення таблиці tournaments, з'явилася нова проблема: країна логічно залежить від міста. Знаючи, що місто Лондон, також відомо, що це Велика Британія, таким чином виходить залежність: tournament_id визначає city, а city визначає country.

Виправлення

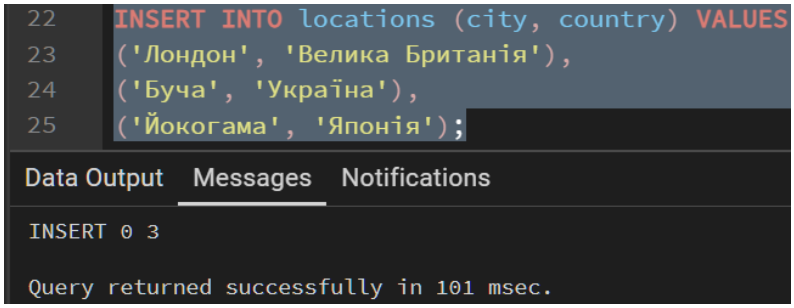
```
CREATE TABLE locations (
location_id SERIAL PRIMARY KEY,
city VARCHAR(100),
country VARCHAR(100)
);
```

Перше, що треба зробити – створити нову таблицю




```
INSERT INTO locations (city, country) VALUES
('Лондон', 'Велика Британія'),
('Буча', 'Україна'),
('Йокогама', 'Японія');
```

Потім треба заповнити таблицю новими даними



```
ALTER TABLE tournaments ADD COLUMN location_id INTEGER REFERENCES
locations(location_id);
```

Тепер створюємо зовнішній ключ до таблиці турнірів

```
26  ALTER TABLE tournaments ADD COLUMN location_id INTEGER REFERENCES locations(location_id);
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 102 msec.
```

```
UPDATE tournaments SET location_id = 1 WHERE city = 'Лондон';

UPDATE tournaments SET location_id = 2 WHERE city = 'Буча';

UPDATE tournaments SET location_id = 3 WHERE city = 'Йокогама';
```

Тепер оновлюємо таблицю турнірів, проставляючи правильні ідентифікатори локацій

```
27  UPDATE tournaments SET location_id = 1 WHERE city = 'Лондон';
28  UPDATE tournaments SET location_id = 2 WHERE city = 'Буча';
29  UPDATE tournaments SET location_id = 3 WHERE city = 'Йокогама';
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 115 msec.
```

```
ALTER TABLE tournaments DROP COLUMN city;

ALTER TABLE tournaments DROP COLUMN country;
```

Тепер видаляємо старі стовпці з таблиці

```
30  ALTER TABLE tournaments DROP COLUMN city;
31  ALTER TABLE tournaments DROP COLUMN country;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 80 msec.
```

```
SELECT * FROM locations;
```

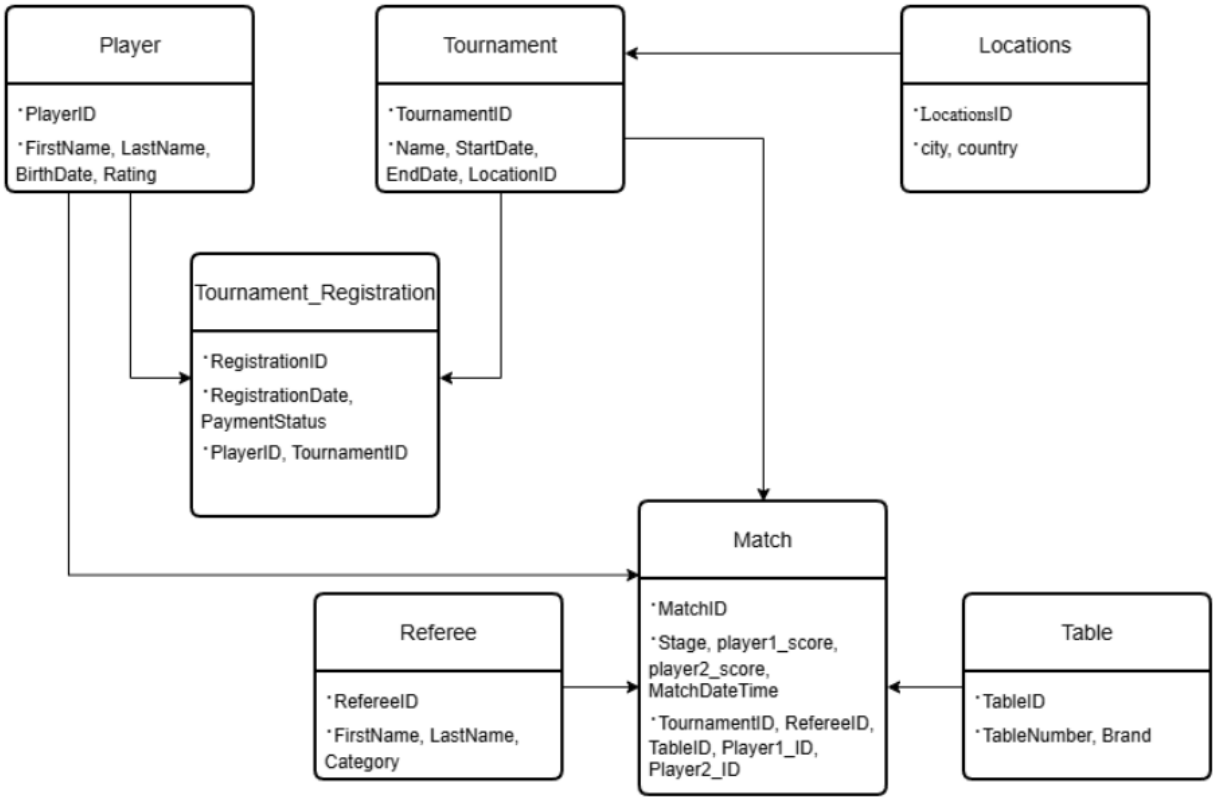
І нарешті перевіряємо як все працює

```
32  SELECT * FROM locations;
```

Data Output | Messages | Notifications

	location_id [PK] integer	city character varying (100)	country character varying (100)
1	1	Лондон	Велика Британія
2	2	Буча	Україна
3	3	Йокогама	Японія

Оновлена діаграма ER



Висновок: під час виконання п'ятої лабораторної роботи я нормалізувала базу даних турнірів з настільного тенісу до третьої нормальної форми (3NF). Для досягнення 1NF я усунула неатомарні значення, розділивши поле location на місто та країну, а загальний рахунок матчу — на бали окремих гравців. Вимоги 2NF виконались автоматично завдяки використанню одиничних первинних ключів, що виключає часткові залежності. Усе виконано успішно, проблем при виконанні не виникло.